UNIVERSITI TUNKU ABDUL RAHMAN

ACADEMIC YEAR 2025/2026

OCTOBER EXAMINATION

UCCD3074 DEEP LEARNING FOR DATA SCIENCE

MONDAY, 6 OCTOBER 2025 TIME : 6.00 PM – 8.00 PM (2 HOURS)

BACHELOR OF COMPUTER SCIENCE (HONOURS)

Instructions to Candidates:

This question paper consists of FIVE (5) questions .

Answer **ALL** questions in **Section A** and **ONLY ONE(1)** question in **Section B**. Each question carries 25 marks.

Should a candidate answers more than ONE (1) question in section B, marks will only be awarded for the FIRST question in that section in the order the candidate submits the answers.

Candidates are allowed to use a calculator.

Answer questions only in the answer booklet provided.

UCCD3074 DEEP LEARNING FOR DATA SCIENCE

SECTION A (Answer all questions)

Q1. (a) Identify the type of neural network that is suitable for each of the following tasks:

(i) Detect the presence of specific keywords in an audio recording. (2 marks)

(ii) Translate a paragraph from English to French. (2 marks)

(iii) Predict the risk level of a loan applicant based on age, income, employment history, and credit score. (2 marks)

(b) Given the following function, gradient descent is used to find the value of **x** that minimizes $f(x)$. The current value $\mathbf{x} = (x_1, x_2) = (3, 4)$.

$$f(x_1, x_2) = x_1^2 + 4x_2^2 - 2x_1x_2 + 3x_1$$

(i) Construct the computational graph for $f(x_1, x_2)$. Use only the following basic computational units: **square** (^), **addition** (+) and **multiplication** (*) all of which supports up to 4 inputs, and **negation** (-) which supports only single input. (7 marks)

(ii) Using backpropagation, compute the gradients $\frac{df}{dx_1}$ and $\frac{df}{dx_2}$ (8 marks)

(iii) Analyze how the presence of the cross term $-2x_1x_2$ affects the cost surface. How does this impact the optimization path of gradient descent? (4 marks)

[Total : 25 marks]

Q2. You are developing a model to predict whether an aerial image contains signs of forest fire (label = 1) or not (label = 0). You collected 1000 images of size 64 × 64 pixels, with 3 colour channels (RGB).

(a) A plain **Convolutional Neural Network (CNN)** is used to handle the task.

(i) What is the shape of the single input in channel-first (PyTorch) format? (3 marks)

(ii) What are the advantages of using a convolutional layer compared to a fully connected layer in this image classification task? (3 marks)

(iii) Assume the first convolutional layer has 16 filters with filter size = 3, with stride = 1 and no padding. How many learnable parameters does this layer have (including biases)? (3 marks)

(iv) Suppose that you decide to enrich the model by incorporating multiple types of imaging (multi-modal) data for each plot, such as RGB aerial images and thermal images. Propose **TWO (2)** different approaches to integrate this multi-modal data into your model. (6 marks)

(b) **ResNet** uses residual blocks rather than the plain block as its basic building block. Figure 2.1 shows a plain block with two convolutional layers.

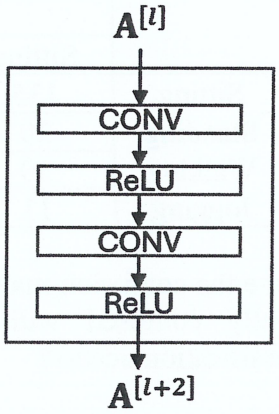

Figure 2.1: A plain block with two CONV layers

(i) Add the residual connection to convert the plain block in Figure 2.1 into a **residual block**. (3 marks)

(ii) Explain how the residual connection can help training. (3 marks)

(iii) The **first** residual block of some ResNet blocks may have different number of channels in $\mathbf{A}^{[l]}$ and $\mathbf{A}^{[l+2]}$. How can we solve this issue? (4 marks)

[Total: 25 marks]

Q3. A wearable watch predicts the bodily postures, namely *standing*, *sitting*, *walking*, and *jogging*. It has two sensor channels representing respiratory and motion data collected from wearable devices. The airflow sensor captures breath patterns, while the accelerometer detects physical movement of the body. A Recurrent Neural Network (RNN) model is trained on sufficient training samples captured from many users.

(a) Suppose that each batch data contains time-series data from 4 users. For each user, data is collected for 60 time steps. The RNN uses the following setting: *num_layers* = 2, *hidden_size* = 8, *bidirectional* = True, and *batch_first* = True.

(i) Justify why an RNN network is suitable for modeling this task. (3 marks)

(ii) Would you use a **many-to-many** or **many-to-one** architecture for the problem? Justify your answer. (3 marks)

(iii) What is the shape of the batch input $\mathbf{X}$ and the output of the RNN layer $\hat{\mathbf{Y}}$? Use PyTorch data format and identify each dimension. (4 marks)

(iv) Design an RNN network to model the task. Show your answer in the form of the block diagram of the unrolled version of the RNN network. (9 marks)

(b) During validation, the confusion matrix is shown as follows:

		Predicted			
		Sitting	Standing	Walking	Jogging
Actual	Sitting	135	50	7	8
	Standing	30	160	4	6
	Walking	30	12	95	63
	Jogging	11	22	65	102

(i) Examine the confusion matrix and identify which two classes are most frequently confused with each other. Why do you think this misclassification occurs? (3 marks)

(ii) Suggest at least **ONE (1)** approach to help the model better distinguish these two classes. (3 marks)

[Total: 25 marks]

SECTION B (Choose ONE question only)

Q4. (a) Compare Recurrent Neural Networks (RNNs) and Transformer architectures with respect to the following aspects. Justify your answer.

(i) Training efficiency and parallelization. (4 marks)

(ii) Ability to capture long-range dependencies. (4 marks)

(iii) Depth of the network. (4 marks)

(b) The self-attention formula of a transformer layer is given below:

$$\hat{\mathbf{V}} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V}$$

Given the following **weights** and **values**:

$$\textbf{weights} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right) = \begin{bmatrix} 0.7 & 0.2 & 0.1 \\ 0.1 & 0.8 & 0.1 \\ 0.2 & 0.2 & 0.6 \end{bmatrix}, \quad \textbf{values} = \mathbf{V} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}$$

(i) Explain the roles of the matrices **Q**, **K** and **V**. (3 marks)

(ii) What is the number of tokens in the input sequence in this self-attention operation? (3 marks)

(iii) Identify the **weights** and the **value** of the first token. (4 marks)

(iv) Compute the new representation (output) of the first token after applying self-attention. (3 marks)

[Total: 25 marks]

Q5. The raw (unnormalized) logits output from a Large Language Model (LLM) for the next token prediction are shown below:

Token	Logit
"cat"	2.0
"dog"	1.0
"car"	0.5
"hat"	0.0
"bat"	-1.0

(a) Temperature decoding is applied to the LLM output.

(i) What is the purpose of temperature decoding? Explain the effect of setting the temperature for each of the following settings:

- $T = 1$
- $T < 1$
- $T > 1$

(5 marks)

(ii) Using temperature = 0.5, compute the softmax probabilities for each token. Show your working and provide probabilities to 3 decimal places. (6 marks)

(b) You are building a chatbot that answers user questions using a Large Language Model. After temperature decoding with $T = 0.5$ in Q5(a)(ii), different sampling strategy can be applied.

(i) Compare how the chatbot's responses would differ if it uses **greedy** decoding (i.e., select the token with the probit value) versus **random** sampling (i.e., sample from the full distribution). In this context, which approach is more appropriate and why? (4 marks)

(ii) Explain **top-k** and **top-p (nucleus)** sampling strategies in LLMs. Then, compare these two strategies. (4 marks)

(iii) Based on your result from Q5(a)(ii), identify the tokens selected under top-k sampling with k=3. (3 marks)

(iv) Based on your result from Q5(a)(ii), suggest a setting of p such that only the top 3 tokens (by probability) are considered. (3 marks)

[Total: 25 marks]

UCCD3074 DEEP LEARNING FOR DATA SCIENCE

Appendix

Logistic Regression

Forward propagation

$z = w_1x_1 + \cdots + w_{F_x}x_{F_x} + b = \mathbf{x}^T\mathbf{w} + b$

$\mathbf{z} = \mathbf{X}\mathbf{w} + b$

$\hat{y} = \sigma(z) = \frac{1}{1+e^{-z}}$

$\hat{\mathbf{y}} = \sigma(\mathbf{z}) = \frac{1}{1+e^{-\mathbf{z}}}$

Gradient Descent

$\mathbf{w} := \mathbf{w} - \alpha\frac{\partial J(\mathbf{w},b)}{\partial \mathbf{w}}$

$b := b - \alpha\frac{\partial J(\mathbf{w},b)}{\partial b}$

$\frac{\partial J}{\partial \hat{y}} = \frac{1}{M}\left(\frac{1-y}{(1-\hat{y})} - \frac{y}{\hat{y}}\right)$

$\frac{\partial J}{\partial z} = \frac{\partial J}{\partial \hat{y}} * (\hat{y} * (1-\hat{y}))$

$\frac{\partial J}{\partial \mathbf{w}} = \mathbf{X}^T\frac{\partial J}{\partial z}$

$\frac{\partial J}{\partial b} = \left(1^T\frac{\partial J}{\partial z}\right)^T$

Standard Neural Network

Forward propagation

$\mathbf{Z}^{[l]} = \mathbf{A}^{[l-1]}\mathbf{W}^{[l]} + \mathbf{b}^{[l]}$

$\mathbf{A}^{[l]} = g(\mathbf{Z}^{[l]})$

$p_i = \frac{e^{z_i}}{\sum_{j=1}^{C} e^{z_j}}$ (softmax)

Gradient Descent

$\frac{\partial J}{\partial \hat{\mathbf{Y}}} = \frac{1}{C\times B}\left(\frac{1-\mathbf{Y}}{1-\hat{\mathbf{Y}}} - \frac{\mathbf{Y}}{\hat{\mathbf{Y}}}\right)$

$\frac{\partial J}{\partial \mathbf{Z}^{[l]}} = \begin{cases} \frac{\partial J}{\partial \mathbf{A}^{[l]}} * \mathbf{A}^{[l]} * (1 - \mathbf{A}^{[l]}) & \text{for sigmoid activation} \\ \frac{\partial J}{\partial \mathbf{A}^{[l]}} * (\mathbf{Z}^{[l]} > 0) & \text{for relu activation} \end{cases}$

$\frac{\partial J}{\partial \mathbf{W}^{[l]}} = \mathbf{A}^{[l-1]T}\frac{\partial J}{\partial \mathbf{Z}^{[l]}}$

$\frac{\partial J}{\partial \mathbf{b}^{[l]}} = \left(1^T\frac{\partial J}{\partial \mathbf{Z}^{[l]}}\right)^T$

$\frac{\partial J}{\partial \mathbf{A}^{[l-1]}} = \frac{\partial J}{\partial \mathbf{Z}^{[l]}}\mathbf{W}^{[l]T}$

Activation Functions

Sigmoid

$g(z) = \frac{1}{1+e^{-z}}, \quad \frac{dg}{dz} = g(z).(1-g(z))$

Tanh

$g(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}, \quad \frac{dg}{dz} = 1 - g(z)^2$

ReLU

$g(z) = \max(0,z), \quad \frac{dg}{dz} = \begin{cases} 1 & \text{if } z > 0 \\ \text{undefined} & \text{if } z = 0 \\ 0 & \text{if } z < 0 \end{cases}$

Leaky ReLU

$g(z) = \max(0.01z,z), \quad \frac{dg}{dz} = \begin{cases} 1 & \text{if } z > 0 \\ \text{undefined} & \text{if } z = 0 \\ 0.01 & \text{if } z < 0 \end{cases}$

Cost Function

Binary cross entropy

$J(\mathbf{W},\mathbf{b}) = \frac{1}{B}\sum_{i=1}^{B} -y^{(i)}\ln\hat{y}^{(i)} - (1-y^{(i)})\ln(1-\hat{y}^{(i)})$

$= \text{mean}(-\mathbf{y} * \ln\hat{\mathbf{y}} - (1-\mathbf{y}) * \ln(1-\hat{\mathbf{y}}))$

Binary cross entropy (multi-label)

$J(\mathbf{W},\mathbf{b}) = \frac{1}{C\times B}\sum_{c=1}^{C}\sum_{i=1}^{B} -y_c^{(i)}\ln\hat{y}_c^{(i)} - \left(1-y_c^{(i)}\right)\ln\left(1-\hat{y}_c^{(i)}\right)$

$= \text{mean}\left(-\mathbf{Y} * \ln\hat{\mathbf{Y}} - (1-\mathbf{Y}) * \ln\left(1-\hat{\mathbf{Y}}\right)\right)$

Negative Log Likelihood

$J(\mathbf{W},\mathbf{b}) = \frac{1}{B}\sum_{i=1}^{B} -\ln\left(p_c^{(i)}\right)$

RNN and LSTM

RNN

$\mathbf{H}^{<t>} = g(\mathbf{H}^{<t-1>}\mathbf{W}_{hh} + \mathbf{X}^{<t>}\mathbf{W}_{hx} + \mathbf{b}^T)$

$\mathbf{H}^{<t>} = g([\mathbf{H}^{<t-1>} \quad \mathbf{X}^{<t>}]\mathbf{W}_h + \mathbf{b}^T)$

LSTM

$\mathbf{F}^{<t>} = \sigma([\mathbf{H}^{<t-1>} \quad \mathbf{X}^{<t>}]\mathbf{W}_f + \mathbf{b}_f^T)$

$\mathbf{I}^{<t>} = \sigma([\mathbf{H}^{<t-1>} \quad \mathbf{X}^{<t>}]\mathbf{W}_i + \mathbf{b}_i^T)$

$\mathbf{O}^{<t>} = \sigma([\mathbf{H}^{<t-1>} \quad \mathbf{X}^{<t>}]\mathbf{W}_o + \mathbf{b}_o^T)$

$\tilde{\mathbf{C}}^{<t>} = \tanh([\mathbf{H}^{<t-1>} \quad \mathbf{X}^{<t>}]\mathbf{W}_c + \mathbf{b}_c^T)$

$\mathbf{C}^{<t>} = \mathbf{I}^{<t>} * \tilde{\mathbf{C}}^{<t>} + \mathbf{F}^{<t>} * \mathbf{C}^{<t-1>}$

$\mathbf{H}^{<t>} = \mathbf{O}^{<t>} * \tanh(\mathbf{C}^{<t>})$

Convolutional Neural Network

$\mathbf{A}^{[l]}(i,j) = \mathbf{A}_{loc}^{[l-1]}(i^*,j^*) \cdot \mathbf{W}^{[l]} + b^{[l]}$

$o = \left\lfloor\frac{i+2p-f}{s} + 1\right\rfloor$

#params in a filter $= c_f \times h_f \times w_f + 1$

#params in a CONV layer = #filters × #params in a filter

#computations = #output activations × #computations per activation

= output volume size × #params in a filter

Efficient Net

depth $d = \alpha^\phi$, width $w = \beta^\phi$, resolution $r = \gamma^\phi$

subject to: $\alpha.\beta^2.\gamma^2 \approx 2$

$\alpha \geq 1, \beta \geq 1, \gamma \geq 1$

Transformers

$\acute{\mathbf{V}} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V}$

$\acute{\mathbf{V}}_m = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T+\mathbf{M}}{\sqrt{d_k}}\right)\mathbf{V}$

UCCD3074 DEEP LEARNING FOR DATA SCIENCE

Appendix

Optimizers

Momentum

$$\mathbf{v}^{<t>} := \beta \mathbf{v}^{<t-1>} + (1-\beta)\frac{\partial J}{\partial \mathbf{w}}^{<t>}$$
$$\mathbf{v}^{<t>} := \frac{1}{1-\beta^t}\mathbf{v}^{<t>}$$
$$\mathbf{w}^{<t>} := \mathbf{w}^{<t-1>} - \alpha \mathbf{v}^{<t>}$$
$$\text{weight}^{<t-n>} = (1-\beta)\beta^n$$

RMSProp

$$\mathbf{s}^{<t>} := \beta \mathbf{s}^{<t>} + (1-\beta)\left(\frac{\partial J}{\partial \mathbf{w}}^{<t>}\right)^2$$
$$\mathbf{w}^{<t>} := \mathbf{w}^{<t-1>} - \left(\frac{\alpha}{\sqrt{\mathbf{s}^{<t>}+\epsilon}}\right)\frac{\partial J}{\partial \mathbf{w}}^{<t>}$$

Adam

$$\mathbf{v}^{<t>} := \beta_1 \mathbf{v}^{<t-1>} + (1-\beta_1)\frac{\partial J}{\partial \mathbf{w}}^{<t>}$$
$$\tilde{\mathbf{v}}^{<t>} = \frac{v^{<t>}}{(1-\beta_1^t)}$$
$$\mathbf{s}^{<t>} := \beta_2 \mathbf{s}^{<t-1>} + (1-\beta_2)\left(\frac{\partial J}{\partial \mathbf{w}}^{<t>}\right)^2$$
$$\tilde{\mathbf{s}}^{<t>} = \frac{s^{<t>}}{(1-\beta_2^t)}$$
$$\mathbf{w}^{<t>} = \mathbf{w}^{<t-1>} - \left(\frac{\alpha}{\sqrt{\tilde{\mathbf{s}}^{<t>}+\epsilon}}\right)\tilde{\mathbf{v}}^{<t>}$$

Learning Rate Decay

$$\alpha^{<t>} = \frac{1}{1+rt}\alpha^{<0>}$$
$$\alpha^{<t>} = \gamma^{t/r}\,\alpha^{<0>}$$
$$\alpha^{<t>} = e^{-k\times t}\,\alpha^{<0>}$$

Initialization

Xavier Initialization

$$var(\mathbf{W}^{[l]}) = \frac{2}{(F^{[l-1]}+F^{[l]})}$$

Kaiming Initialization

$$var(\mathbf{W}^{[l]}) = \frac{2}{F^{[l-1]}}$$

Normalization

BatchNorm

$$\hat{\mathbf{Z}}[:,f] = \frac{\mathbf{Z}[:,f]-\mu_f}{\sqrt{v_f+\epsilon}}$$
$$\hat{\mathbf{Z}}[:,c,:,:] = \frac{\mathbf{Z}[:,c,:,:]-\mu_c}{\sqrt{v_c+\epsilon}}$$
$$\tilde{\mathbf{Z}}[:,c,:,:] = \gamma_c \hat{\mathbf{Z}}[:,c,:,:] + \beta_c$$
$$\mu_c^{\{t\}} = \text{mean}\left(\mathbf{Z}^{\{t\}}[:,c,:,:]\right) \quad \text{(training mode)}$$
$$v_c^{\{t\}} = \text{variance}\left(\mathbf{Z}^{\{t\}}[:,c,:,:]\right) \quad \text{(training mode)}$$
$$\vec{\mu}_c = (1-m)\,\vec{\mu}_c + m\mu_c^{\{t\}} \quad \text{(eval mode)}$$
$$\vec{v}_c = (1-m)\,\vec{v}_c + m v_c^{\{t\}} \quad \text{(eval mode)}$$

LayerNorm

$$\hat{\mathbf{Z}}[i,t,:] = \frac{(\mathbf{Z}[i,t,:]-\mu^{(i)<t>})}{\sqrt{v^{(i)<t>}+\epsilon}}$$
$$\tilde{\mathbf{Z}}[i,t,:] = \gamma^{(i)<t>}\hat{\mathbf{Z}}[i,t,:] + \beta^{(i)<t>}$$
$$\mu^{(i)<t>} = \text{mean}\,(\mathbf{Z}[i,t,:]) \quad \text{(training and eval model)}$$
$$v^{(i)<t>} = \text{variance}\,(\mathbf{Z}[i,t,:]) \quad \text{(training and eval model)}$$

Sampling Strategy

$$z_i' = \frac{z_i}{\text{repeat}_{\text{count}} * \text{penalty\_factor}} \quad \text{(repeat penalty)}$$

$$p'(w_i) = \frac{\exp\left(\frac{z_i}{T}\right)}{\sum_j \exp\left(\frac{z_j}{T}\right)} \quad \text{(temperature decoding)}$$

LLM Evaluation

$$\text{prec}(\hat{\mathbf{y}}) = \sum_i \frac{\text{exist\_in\_any\_ref}(\hat{y}_i)}{|\hat{y}|} \quad \text{(Standard precision)}$$

$$p_n = \frac{\sum_{w\in ngram}\min\left(\text{count}_{\text{LLM}}(w),\max\left(\text{count}_{\text{gt}}(w)\right)\right)}{\sum_{w\in ngram}\text{count}_{\text{LLM}}(w)} \quad \text{(Modified precision)}$$

$$\text{bleu score} = \beta\exp\left(\sum_{n=1}^{N} w_n \ln p_n\right) \quad \text{(Bleu score)}$$